

■ PANDUAN LENGKAP PRAKTIKUM 2 & 4

Python Windows - Step by Step untuk Taruna

PERSIAPAN AWAL (WAJIB - Lakukan Sekali Saja)

Step 0.1: Install Python

1. Buka browser → kunjungi <https://www.python.org/downloads/>
2. Download Python 3.12 atau lebih baru
3. **PENTING:** Saat install, centang ☒ “Add Python to PATH”
4. Klik “Install Now” dan tunggu selesai

Step 0.2: Verifikasi Python Terinstall

Buka **Command Prompt** (CMD) atau **PowerShell**:

```
python --version
```

Jika muncul Python 3.12.x →  Berhasil

Step 0.3: Buat Folder Praktikum

```
mkdir C:\Praktikum_Kriptografi  
cd C:\Praktikum_Kriptografi
```

Step 0.4: Install Library Python (SEKALI SAJA)

```
pip install cryptography requests py7zr matplotlib
```

Tunggu sampai semua terinstall. Jika ada error, coba satu per satu:

```
pip install cryptography  
pip install requests  
pip install py7zr  
pip install matplotlib
```

Step 0.5: Verifikasi Library

```
python -c "import cryptography; print('OK cryptography')"  
python -c "import requests; print('OK requests')"  
python -c "import py7zr; print('OK py7zr')"  
python -c "import matplotlib; print('OK matplotlib')"
```

Jika semua muncul “OK” →  Siap praktikum!

DAFTAR FILE YANG HARUS DIBUAT

Salin setiap file berikut ke folder C:\Praktikum_Kriptografi menggunakan Notepad/VS Code.

No	Nama File	Fungsi
1	01_aes_encryption.py	Enkripsi/Dekripsi AES-256-CBC
2	02_rsa_keygen.py	Generate pasangan kunci RSA
3	03_rsa_encrypt.py	Enkripsi/Dekripsi RSA-OAEP
4	04_encrypt_hybrid.py	Hybrid Encryption (Encrypt)
5	05_decrypt_hybrid.py	Hybrid Encryption (Decrypt)
6	06_benchmark.py	Benchmark perbandingan
7	07_https_analyzer.py	Analisis sertifikat HTTPS
8	08_http_server.py	HTTP Server (Tidak Aman)
9	09_https_server.py	HTTPS Server (Aman)
10	10_zip_encrypt.py	Enkripsi ZIP vs 7z
11	11_selfsigned_cert.py	Generate Self-Signed Certificate
12	12_https_server_custom.py	HTTPS Server dengan Self-Signed Cert

PRAKTIKUM 2: KRIPTOGRAFI SIMETRIS & ASIMETRIS

✓ LANGKAH A: Enkripsi File dengan AES-256-CBC

Tujuan: Mengenkripsi dan mendekripsi file menggunakan AES-256-CBC dengan PBKDF2

Cara Kerja: 1. Buat file `rahasia.txt` dengan isi “Data Rahasia Siber” 2. Enkripsi file menggunakan password + PBKDF2 + AES-256-CBC 3. Dekripsi file menggunakan password yang sama 4. Verifikasi hasil dekripsi identik dengan file asli

Perintah:

```
cd C:\Praktikum_Kriptografi
python 01_aes_encryption.py
```

Output yang Diharapkan:

```
=====
[STEP 1] Generate SALT acak (16 byte)
        Salt (hex): a1b2c3d4...

[STEP 2] Derive KEY dari password menggunakan PBKDF2
        - Algoritma: SHA-256
        - Iterasi: 100,000
        - Panjang Key: 32 byte (256-bit)

[STEP 3] Generate IV (Initialization Vector) acak (16 byte)
        IV (hex): e5f6g7h8...

[STEP 4] Baca file 'rahasia.txt'
        Ukuran plaintext: 18 byte

[STEP 5] Padding PKCS7
        Padding: 14 byte

[STEP 6] Enkripsi dengan AES-256-CBC
        Ukuran ciphertext: 32 byte

[STEP 7] Simpan: SALT + IV + CIPHERTEXT ke 'rahasia.enc'
        Total ukuran file: 64 byte
=====
```

[✓] BERHASIL: 'rahasia.txt' dienkripsi ke 'rahasia.enc'

=====

File yang Dihasilkan: - rahasia.txt → File asli - rahasia.enc → File terenkripsi (binary, tidak bisa dibaca) - hasil.txt → File hasil dekripsi (identik dengan asli)

Screenshot yang Diperlukan: - Screenshot terminal saat enkripsi berhasil - Screenshot terminal saat dekripsi berhasil - Screenshot isi file rahasia.txt dan hasil.txt yang identik

✓ LANGKAH B: Membuat Pasangan Kunci RSA

Tujuan: Generate public key dan private key RSA 2048-bit

Cara Kerja: 1. Generate private key RSA 2048-bit 2. Ekstrak public key dari private key 3. Simpan ke file private.pem dan public.pem

Perintah:

```
python 02_rsa_keygen.py
```

Output yang Diharapkan:

=====

```
[STEP 1] Generate private key RSA
        Ukuran: 2048 bit
        Public exponent: 65537 (standar F4)
        [✓] Private key berhasil dibuat
```

```
[STEP 2] Simpan private key ke 'private.pem'
        [✓] Private key tersimpan
```

```
[STEP 3] Simpan public key ke 'public.pem'
        [✓] Public key tersimpan
```

[ISI KUNCI]

=====

```
--- PRIVATE KEY (RAHASIA - JANGAN DIBAGIKAN!) ---
```

```
-----BEGIN PRIVATE KEY-----
```

```
MIIEvQIBADANBgkqhkiG9w0BAQEFAASCBAKcwgSjAgEAAoIBAQC...
```

```
...
```

```
-----END PRIVATE KEY-----
```

```
--- PUBLIC KEY (BOLEH DIBAGIKAN) ---
```

```
-----BEGIN PUBLIC KEY-----
```

```
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEA...
```

...
-----END PUBLIC KEY-----

File yang Dihasilkan: - private.pem → Private key (RAHASIA!) - public.pem → Public key (boleh dibagikan)

Screenshot yang Diperlukan: - Screenshot terminal saat pembuatan kunci berhasil - Screenshot isi file private.pem dan public.pem

✅ LANGKAH C: Enkripsi & Dekripsi Pesan dengan RSA-OAEP

Tujuan: Mengenkripsi pesan dengan public key, mendekripsi dengan private key

Cara Kerja: 1. Buat file pesan.txt dengan isi "Pesan Rahasia untuk AKMIL" 2. Enkripsi dengan public key menggunakan RSA-OAEP 3. Dekripsi dengan private key 4. Verifikasi pesan identik

Perintah:

```
python 03_rsa_encrypt.py
```

Output yang Diharapkan:

```
=====
[STEP 1] Baca public key dari file
        [✓] Public key loaded dari 'public.pem'

[STEP 2] Baca plaintext dari 'pesan.txt'
        Ukuran plaintext: 27 byte
        Isi: 'Pesan Rahasia untuk AKMIL'

[STEP 3] Enkripsi dengan RSA-OAEP
        - Padding: OAEP
        - Hash: SHA-256
        - MGF: MGF1-SHA256

[STEP 4] Simpan ciphertext ke 'pesan.enc'
        Ukuran ciphertext: 256 byte

=====
[✓] BERHASIL: Pesan dienkripsi!
=====

=====
[STEP 1] Baca private key dari file
        [✓] Private key loaded dari 'private.pem'
```

[STEP 2] Baca ciphertext dari 'pesan.enc'
Ukuran ciphertext: 256 byte

[STEP 3] Dekripsi dengan RSA-OAEP

[STEP 4] Simpan plaintext ke 'hasil_deskripsi.txt'
Isi: 'Pesan Rahasia untuk AKMIL'

=====
[✓] BERHASIL: Pesan didekripsi!
=====

File yang Dihasilkan: - pesan.txt → Pesan asli - pesan.enc → Pesan terenkripsi (binary) - hasil_deskripsi.txt → Hasil dekripsi (identik dengan asli)

Screenshot yang Diperlukan: - Screenshot terminal enkripsi RSA - Screenshot terminal dekripsi RSA - Screenshot isi file pesan.txt dan hasil_deskripsi.txt

✓ LANGKAH D: Tabel Perbandingan

Isi tabel berikut berdasarkan hasil praktikum:

Aspek	Simetris (AES)	Asimetris (RSA)
Jumlah Key	1 key (shared secret)	2 key (public + private)
Kecepatan	Sangat cepat (~GB/s)	Sangat lambat (~MB/s)
Proses		
Distribusi Key	Sulit (harus aman)	Mudah (public bebas dibagikan)
Keamanan	Bergantung secrecy key	Matematis (faktorisasi prima)
Penggunaan Umum	Enkripsi data/file besar	Distribusi kunci, digital signature
Ukuran Data	Tidak terbatas	Maks ~190 byte (RSA 2048)
Contoh Algoritma	AES-256, ChaCha20	RSA-2048/4096, ECC P-256

✓ LANGKAH E: Jawaban Pertanyaan Dasar

1. Perbedaan utama simetris vs asimetris: - **Simetris:** Satu kunci sama untuk enkripsi & dekripsi. Cepat tapi distribusi kunci sulit. - **Asimetris:** Pasangan kunci (public/private). Lambat tapi distribusi kunci mudah.

6. Password salah pada AES: - Terjadi error atau output karakter acak (gibberish). - PBKDF2 dengan 100,000 iterasi: setiap percobaan butuh 100,000 operasi hash. Brute-force 1 juta password = 100 miliar operasi → sehari-hari/bertahun-tahun.

7. Mengapa RSA lebih lambat: - **Matematis:** Kompleksitas $O(k^3)$ untuk eksponensiasi modular. RSA-2048 butuh operasi dengan bilangan 2048-bit. -

Komputasional: AES bekerja pada block 128-bit dengan substitution-permutation (cepat di hardware). RSA butuh modular exponentiation pada bilangan besar.

8. Situasi penggunaan: - **AES:** Enkripsi file database, streaming video, disk encryption (BitLocker). - **RSA:** Enkripsi kunci AES (hybrid), digital signature sertifikat SSL, secure email (PGP).

✓ LANGKAH F: Hybrid Encryption Shell Script (Python)

Tugas 1: encrypt_hybrid.py

Cara Kerja: 1. Generate RSA 4096-bit key pair 2. Buat file secret.txt 3. Generate random AES password (256-bit) 4. Encrypt file dengan AES-256-CBC 5. Encrypt AES password dengan RSA public key 6. Cleanup plaintext password

Perintah:

```
python 04_encrypt_hybrid.py
```

Output yang Diharapkan:

```
=====
[STEP 1] Generate RSA 4096-bit key pair
    [✓] private.pem tersimpan
    [✓] public.pem tersimpan

=====
[STEP 2] Generate random AES session key (256-bit)
    Key (hex): a1b2c3d4...
```

[STEP 3] Generate random IV (128-bit)
IV (hex): e5f6g7h8...

[STEP 4] Baca file 'secret.txt'
Ukuran: 470 byte

[STEP 5] Enkripsi file dengan AES-256-CBC
[✓] File terenkripsi: pesan_rahasia.enc

[STEP 6] Enkripsi AES key dengan RSA public key
[✓] AES key terenkripsi: aes_key.enc

[STEP 7] Secure cleanup
[✓] AES key dihapus dari memory

=====
[✓] HYBRID ENCRYPTION BERHASIL!
=====

Output files:

- pesan_rahasia.enc → File terenkripsi (AES)
- aes_key.enc → Session key terenkripsi (RSA)
- public.pem → Public key RSA
- private.pem → Private key RSA (RAHASIA!)

Tugas 2: decrypt_hybrid.py

Perintah:

python 05_decrypt_hybrid.py

Output yang Diharapkan:

=====
[CHECK] Memeriksa prasyarat file...
[✓] private.pem (3243 byte)
[✓] aes_key.enc (512 byte)
[✓] pesan_rahasia.enc (480 byte)
[✓] Semua prasyarat terpenuhi

=====
[STEP 1] Decrypt AES key dengan RSA private key
[✓] Private key loaded
[✓] AES key berhasil didekripsi

[STEP 2] Decrypt file dengan AES-256-CBC
[✓] File berhasil didekripsi: hasil_decrypt.txt

[STEP 3] Cleanup temporary files
[✓] AES key dihapus dari memory

=====

[✓] HYBRID DECRYPTION BERHASIL!

=====

[VERIFIKASI] Isi file hasil dekripsi:

Ini adalah pesan rahasia yang sangat panjang...

Tugas 4: Perhitungan RSA untuk file 50MB

Ukuran file: 50MB = 52,428,800 byte
Ukuran block RSA-2048: ~190 byte
Jumlah operasi RSA: $52,428,800 / 190 = 275,941$ operasi
Estimasi waktu: $275,941 \times 0.01$ detik = 2,759 detik $\approx$ 46 menit

Hybrid encryption: 1 operasi RSA + 1 operasi AES
= ~ 0.01 detik + ~ 0.1 detik = ~ 0.11 detik

Kecepatan hybrid: $275,941 / 1 = 275,941$ x lebih cepat!

LANGKAH G: Benchmark Hybrid Encryption

Tujuan: Bandingkan kecepatan AES vs RSA vs Hybrid untuk file 10MB, 50MB, 100MB

Perintah:

python 06_benchmark.py

Output yang Diharapkan:

```
--- Testing 10MB ---
AES: 0.0234s
RSA (simulated): 526.32s (52,632 blocks)
Hybrid: 0.0241s

--- Testing 50MB ---
AES: 0.1156s
RSA (simulated): 2631.58s (263,158 blocks)
Hybrid: 0.1162s

--- Testing 100MB ---
AES: 0.2312s
RSA (simulated): 5263.16s (526,316 blocks)
Hybrid: 0.2321s
```

```
=====
```

HASIL BENCHMARK			
=====			
Ukuran	AES	RSA (estimasi)	Hybrid

10MB	0.0234	526.32	0.0241
50MB	0.1156	2631.58	0.1162
100MB	0.2312	5263.16	0.2321

[✓] Grafik disimpan: benchmark.png

Screenshot yang Diperlukan: - Screenshot terminal hasil benchmark -
Screenshot file benchmark.png (grafik perbandingan)

PRAKTIKUM 4: KEAMANAN SIBER

LANGKAH A: Analisis HTTPS pada Browser (Python)

Tujuan: Menganalisis sertifikat digital dari website HTTPS

Perintah:

```
python 07_https_analyzer.py
```

Output yang Diharapkan:

```
=====
ANALISIS HTTPS: https://www.google.com
=====
```

```
[1] Melakukan HTTPS request...
```

```
    [✓] Status: 200
```

```
    [✓] Server: gws
```

```
[2] Menganalisis sertifikat SSL/TLS...
```

```
-----
CIPHER SUITE
```

```
-----
    Protocol Version: TLSv1.3
    Cipher: TLS_AES_256_GCM_SHA384
    Bits: 256
```

```
-----
CERTIFIKAT DIGITAL
-----
```

- a) SUBJECT / COMMON NAME (CN):
commonName: *.google.com
- b) SUBJECT ALTERNATIVE NAME (SAN):
DNS: *.google.com
DNS: google.com
DNS: *.youtube.com
...
- c) ISSUER (Certificate Authority):
commonName: GTS CA 1C3
organizationName: Google Trust Services
- d) VALIDITY PERIOD:
Not Before: Feb 10 00:00:00 2026 GMT
Not After: May 5 00:00:00 2026 GMT
- e) SERIAL NUMBER: 1234567890ABCDEF...

[✓] ANALISIS SELESAI

Screenshot yang Diperlukan: - Screenshot terminal analisis 3 website (Google, GitHub, Universitas) - Screenshot browser: klik ikon gembok → Certificate → Details

✓ LANGKAH B: Analisis Paket dengan Wireshark

Catatan: Wireshark memerlukan hak akses administrator.

Langkah-langkah: 1. Install Wireshark dari <https://www.wireshark.org/> 2. Buka Wireshark sebagai Administrator 3. Pilih interface jaringan (Wi-Fi atau Ethernet) 4. Mulai capture 5. Filter paket: `tls.handshake.type == 1` # Client Hello
`tls.handshake.type == 2` # Server Hello `tls.handshake.type == 11`
Certificate 6. Buka browser, akses <https://www.google.com> 7. Amati proses TLS Handshake

Screenshot yang Diperlukan: - Screenshot Wireshark: Client Hello - Screenshot Wireshark: Server Hello - Screenshot Wireshark: Certificate Exchange - Screenshot Wireshark: Encrypted Application Data

✅ LANGKAH C: Simulasi HTTP vs HTTPS

Langkah 1: HTTP Server (Tidak Aman)

Buka terminal 1:

```
python 08_http_server.py
```

Buka browser, akses: <http://localhost:8080> Isi form login dan submit.

Output di terminal:

```
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
[!] DATA CAPTURED (HTTP/PLAINTEXT):
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
    Raw data: user=admin&pass=password123
    Username: admin
    Password: password123
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
```

Langkah 2: HTTPS Server (Aman)

Buka terminal 2:

```
python 09_https_server.py
```

Buka browser, akses: <https://localhost:4443> Browser akan menampilkan peringatan “Not Secure” karena self-signed. Klik “Advanced” → “Proceed to localhost”.

Screenshot yang Diperlukan: - Screenshot browser form HTTP - Screenshot terminal dengan data plaintext - Screenshot browser HTTPS dengan peringatan - Screenshot perbedaan Wireshark HTTP vs HTTPS

✅ LANGKAH D: Enkripsi File dengan ZIP Password

Perintah:

```
python 10_zip_encrypt.py
```

Output yang Diharapkan:

```
=====
[ZIP ENCRYPTION - ZipCrypto (LEMAH)]
=====
    [✓] Ditambahkan: file1.txt
    [✓] Ditambahkan: file2.txt

[!] PERINGATAN: File 'rahasia.zip' dibuat TANPA enkripsi!
    Python zipfile tidak mendukung enkripsi password.
```

Untuk enkripsi ZIP dengan password di Windows, gunakan:
7z a -p rahasia.zip file1.txt file2.txt

ZipCrypto sudah LEMAH dan bisa dipecahkan!

```
=====
[7Z ENCRYPTION - AES-256 (AMAN)]
=====
```

```
[✓] Ditambahkan: file1.txt
[✓] Ditambahkan: file2.txt
```

```
[✓] Arsip 'rahasia.7z' berhasil dibuat!
    Enkripsi: AES-256
    Header encryption: Enabled
    Kompresi: LZMA2
```

```
=====
[EKSTRAKSI] rahasia.7z
=====
```

```
[✓] Berhasil diekstrak ke folder 'extracted'
```

```
=====
[TEST] Coba ekstrak dengan password SALAH
=====
```

```
[X] Gagal! Password salah atau file corrupt.
```

Screenshot yang Diperlukan: - Screenshot terminal ZIP vs 7z - Screenshot hasil ekstraksi berhasil - Screenshot hasil ekstraksi gagal (password salah)

✓ LANGKAH F: Self-Signed Certificate

Perintah:

```
python 11_selfsigned_cert.py
```

Output yang Diharapkan:

```
=====
[STEP 1] Generate RSA 4096-bit private key
    [✓] Private key generated
```

```
[STEP 2] Buat konfigurasi Subject & SAN
    [✓] Subject configured:
        CN: akademik.local
        O: AKMIL
    [✓] SAN configured:
        DNS.1 = akademik.local
```

```
DNS.2 = www.akademik.local
DNS.3 = localhost
IP.1  = 127.0.0.1
```

```
[STEP 3] Build self-signed certificate
[✓] Certificate signed with SHA-256
[✓] Valid for 365 days
```

```
[STEP 4] Simpan ke file
[✓] key.pem (private key)
[✓] cert.pem (certificate)
[✓] akademik.pfx (untuk import ke Windows)
```

```
[✓] SELF-SIGNED CERTIFICATE BERHASIL DIBUAT!
```

```
[INFORMASI SERTIFIKAT]
```

```
-----
Subject:    CN=akademik.local,O=AKMIL,C=ID
Issuer:     CN=akademik.local,O=AKMIL,C=ID
Serial:     123456789...
Not Before: 2026-06-03 05:20:00
Not After:  2027-06-03 05:20:00
Signature:  SHA-256
Key Size:   4096-bit
```

Jalankan HTTPS Server:

```
python 12_https_server_custom.py
```

Buka browser, akses: <https://localhost:4443>

Screenshot yang Diperlukan: - Screenshot terminal generate certificate -

Screenshot browser dengan peringatan "Your connection is not private" -

Screenshot detail sertifikat di browser

RINGKASAN PERINTAH

Perintah	Fungsi
<code>python 01_aes_encryption.py</code>	Enkripsi/Dekripsi AES-256-CBC
<code>python 02_rsa_keygen.py</code>	Generate kunci RSA
<code>python 03_rsa_encrypt.py</code>	Enkripsi/Dekripsi RSA-OAEP
<code>python 04_encrypt_hybrid.py</code>	Hybrid Encryption

Perintah	Fungsi
<code>python 05_decrypt_hybrid.py</code>	Hybrid Decryption
<code>python 06_benchmark.py</code>	Benchmark perbandingan
<code>python 07_https_analyzer.py</code>	Analisis HTTPS
<code>python 08_http_server.py</code>	HTTP Server (port 8080)
<code>python 09_https_server.py</code>	HTTPS Server (port 4443)
<code>python 10_zip_encrypt.py</code>	Enkripsi ZIP/7z
<code>python 11_selfsigned_cert.py</code>	Generate Self-Signed Cert
<code>python 12_https_server_custom.py</code>	HTTPS Server Custom

TIPS MENGERJAKAN LAPORAN

1. **Screenshot setiap langkah** - Jangan lupa screenshot terminal dan browser
 2. **Simpan semua file hasil** - rahasia.enc, pesan.enc, private.pem, dll
 3. **Jawaban singkat tapi tepat** - Langsung jawab inti pertanyaan
 4. **Verifikasi hasil** - Pastikan data asli = data hasil dekripsi
 5. **Format laporan**: Arial 12pt, spasi 1.5, margin normal
 6. **Nama file**: Nama_NoAK_Laporan_Praktikum_X.pdf
-

TROUBLESHOOTING

Error: “ModuleNotFoundError: No module named ‘cryptography’” → Install ulang: `pip install cryptography`

Error: “Permission denied” saat buat file → Jalankan CMD sebagai Administrator

Error: “Address already in use” saat run server → Port sedang dipakai. Ganti port atau tutup program lain.

Error: “bad decrypt” saat dekripsi AES → Password salah! Pastikan password sama saat enkripsi dan dekripsi.

Browser tidak bisa akses <https://localhost:4443> → Pastikan server berjalan.
Cek firewall Windows.

Selamat mengerjakan praktikum! Semoga sukses!